

Dependency Chain Visualization

```
Step 1: Create DOM
  ↓
Step 2: Register Services
  ↓
Step 3: Get eventBus, shapeRegistry
  ↓
Step 4: Load Shapes into shapeRegistry
  ↓
Step 5: Setup UI (LeftPalette uses shapeRegistry)
  ↓
Step 6: Get editor, managers
  ↓
Step 7: Setup Event Handlers (uses all managers)
```

Technically valid alternative order

```
// Option A: Current order (shown in code)
this.createDOMStructure();
ServiceProvider.register(this.container);
this.eventBus = this.container.get("eventBus");
this.shapeRegistry = this.container.get("shapeRegistry");
ShapeLoader.loadBuiltInShapes(this.shapeRegistry);
this.setupUI();

// Option B: Register services first
ServiceProvider.register(this.container);
this.createDOMStructure(); // ← moved here
this.eventBus = this.container.get("eventBus");
this.shapeRegistry = this.container.get("shapeRegistry");
ShapeLoader.loadBuiltInShapes(this.shapeRegistry);
this.setupUI();
```

Best Practice

```
// Step 1: Create DOM structure first
this.createDOMStructure();

// Step 2: Register all services
ServiceProvider.register(this.container);
```

Reasons:

1. **Clarity** — You prepare the canvas (DOM) first, then populate it with services
2. **Psychology** — DOM structure first makes logical sense (prepare containers before populating)
3. **Debugging** — If DOM creation fails, you catch it early before setting up complex services
4. **Consistency** — Creates a clear mental model: "prepare, then populate"

What Really Matters

The **true dependencies** are:

```
Step 4: Load Shapes
↑ Requires shapeRegistry (from Step 3)
```

```
Step 5: Setup UI
↑ Requires DOM (from Step 1)
↑ Requires shapeRegistry (from Step 4)
↑ Requires services like eventBus
```

```
Step 7: Setup Event Handlers
↑ Requires DOM (from Step 1)
↑ Requires all managers (from Step 6)
```

Async, await and sync functions

A function only needs to be async if it performs an asynchronous operation (like fetching a file, reading a database, or using `setTimeout`).

Functions that just do synchronous work (like creating a div, adding an event listener, or creating a new object) should be regular, non-async functions.

Parallel vs. Sequential (The Simple Answer)

This is the key to your question:

To run a function sequentially: Use `await` on it inside your `async initialize()`. This forces the code to wait for it to finish, whether it's async or not.

To run a function in parallel: Call an async function without `await`. This is a "fire and forget" call.

Calling a regular (non-async) function always runs sequentially. Your code will wait for it to finish before moving on, even without `await`, because it's "blocking" by nature.